Transient Heat Conduction in Python

A Student Guide to Explicit and Implicit Methods
and Changing Boundary Conditions

Companion to the two one-dimensional example scripts

What you will learn

This guide connects the heat equation to the calculations in `transient_1d_explicit.py` and `transient_1d_implicit.py`. No previous Python programming experience is assumed.

By the end of the guide, you should be able to:

- interpret the arrays, indexing, and time-stepping loops used in both scripts;
- explain why the explicit method updates temperatures directly while the implicit method solves a system of simultaneous equations; and
- replace a fixed-temperature boundary with a specified heat flux, insulation, or convection boundary condition.

Use this guide with copies of the scripts. The programs currently hold the left end at 100 °C and the right end at 20 °C. The worked boundary changes replace the right condition while retaining the fixed left temperature. These boundary recipes are alternatives, so apply one recipe at a time. The original scripts are not modified by this document.

1. Run a script and find the inputs

Install Python 3 from https://www.python.org/downloads/ if needed. On Windows, select the installer option to add Python to PATH. Save both Python files in a folder. Open that folder in File Explorer, type `cmd` in its address bar, and press Enter. In that command window, install the packages once and run either script:

```
python -m pip install numpy matplotlib
python transient_1d_explicit.py
python transient_1d_implicit.py
```

Run the commands one at a time when comparing timings. Each program prints its solver time and saves a contour PNG beside the script; no plot window opens. The batch launcher runs both programs concurrently, which can affect timings. Edit code in a plain-text editor, save it, and rerun the command.

The editable inputs are near the top of each script. Use the same physical inputs and grid in both files for a direct comparison. The current settings are $k = 45$ W/(m K), $\rho = 7800$ kg/m^3, $c_p = 500$ J/(kg K), $L = 0.10$ m, 201 nodes, $\Delta t = 0.005$ s, and a final time of 300 s. For quicker practice with boundary changes, use 51 nodes and `dt = 0.1` in both copies; this also satisfies the original explicit stability limit.

2. The physical equation and the grid

For a uniform rod with constant properties, no internal heat generation, and no heat loss through its sides,

$$\rho c_p \frac{\partial T}{\partial t} = k \frac{\partial^2 T}{\partial x^2}, \qquad \frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}, \qquad \alpha = \frac{k}{\rho c_p}. \tag{1}$$

The left side describes heat storage; the right side describes the net effect of conduction. Thermal diffusivity α has units of m^2/s. Larger diffusivity makes temperature differences spread through the rod more quickly.

Let $N = \texttt{number_of_nodes} - 1$. Nodes are numbered $i = 0, 1, \ldots, N$, so there are $N + 1$ nodes but only N intervals:

$$x_i = i\Delta x, \quad \Delta x = L/N, \qquad t_n = n\Delta t, \qquad T_i^n \approx T(x_i, t_n).$$

The superscript n labels a time; it is *not* a power. The subscript i labels a position. Interior nodes have $1 \le i \le N - 1$. The dimensionless Fourier number is

$$\mathrm{Fo} = \frac{\alpha \Delta t}{\Delta x^2}.$$

With the current inputs, $\alpha \approx 1.15385 \times 10^{-5}\ \mathrm{m}^2/\mathrm{s}$, $\Delta x = 0.0005$ m, and $\mathrm{Fo} \approx 0.230769$.

How the symbols become Python

```
alpha = k / (rho * cp)          # * multiplies; / divides
dx = L / (number_of_nodes - 1)
Fo = alpha * dt / dx**2         # **2 means squared
x = np.linspace(0.0, L, number_of_nodes)
time = np.arange(number_of_steps + 1) * dt
T_history = np.zeros((number_of_steps + 1, number_of_nodes))
```

`np` is the short name given to NumPy. An array is an organized collection of numbers. Here, each row of `T_history` is a saved time and each column is a position. Thus T_i^n is `T_history[n, i]`. There are 60,001 rows and 201 columns in the current full history, including time zero. The temperature array alone occupies about 96 MB; plotting uses additional memory.

Python expression	Meaning
`T_history[0, :]`	Initial temperatures at all positions
`T_history[n, :]`	All temperatures at saved time n
`T_history[:, 0]`	Left-end temperature at every time
`T_history[:, -1]`	Right-end temperature at every time
`T_history[n, -2]`	Node next to the right end at time n

Python starts counting at zero. A colon selects all entries along an array direction. Negative indices count backward: -1 is last, -2 is next to last. Do not confuse Python index `-1` with a ghost node.

3. Explicit FTCS: calculate from the old temperatures

Approximate the spatial curvature by the centered difference

$$\left.\frac{\partial^2 T}{\partial x^2}\right|_{i,n} \approx \frac{T_{i-1}^n - 2T_i^n + T_{i+1}^n}{\Delta x^2}.$$

Use a forward difference for the time derivative and evaluate the curvature at the old time n:

$$\frac{T_i^{n+1} - T_i^n}{\Delta t} = \alpha \frac{T_{i-1}^n - 2T_i^n + T_{i+1}^n}{\Delta x^2}.$$

Rearranging gives the forward-time, centered-space (FTCS) update:

$$\boxed{T_i^{n+1} = T_i^n + \mathrm{Fo}(T_{i-1}^n - 2T_i^n + T_{i+1}^n).} \tag{2}$$

Every quantity on the right is known. For example, with $\mathrm{Fo} = 0.25$ and old left, center, and right temperatures $80, 40, 20\,^\circ\mathrm{C}$, the new center temperature is $40 + 0.25(80 - 80 + 20) = 45\,^\circ\mathrm{C}$.

```
for n in range(number_of_steps):
    for i in range(1, number_of_nodes - 1):
        T_history[n + 1, i] = T_history[n, i] + Fo * (
            T_history[n, i - 1]
            - 2.0 * T_history[n, i]
            + T_history[n, i + 1]
        )
```

`for` repeats the indented instructions. `range(a, b)` produces integers starting at `a` and stopping *before* `b`. The outer loop advances time. The inner loop updates interior nodes only. Indentation is part of Python's syntax: use four spaces per level. The right side reads row `n`; the left side writes row `n + 1`. Changing row `n` in place would mix old and new temperatures and would no longer be this method.

Why the time step is limited

Equation (2) can be written

$$T_i^{n+1} = \mathrm{Fo}T_{i-1}^n + (1 - 2\mathrm{Fo})T_i^n + \mathrm{Fo}T_{i+1}^n.$$

For $0 \le \mathrm{Fo} \le 1/2$, the weights are nonnegative and sum to one: the update is a weighted average. The standard one-dimensional FTCS scheme with fixed end temperatures requires

$$\mathrm{Fo} \le \tfrac{1}{2}, \qquad \Delta t \le \frac{\Delta x^2}{2\alpha}.$$

The scripts stop if this condition is violated. Halving Δx requires reducing Δt by a factor of four to keep the same Fourier number. Refining the grid therefore increases both the nodes per step and the number of steps. A convection boundary introduces an additional restriction described later.

4. Implicit Backward Euler: solve for the new temperatures

Backward Euler evaluates spatial curvature at the *new* time:

$$\frac{T_i^{n+1} - T_i^n}{\Delta t} = \alpha \frac{T_{i-1}^{n+1} - 2T_i^{n+1} + T_{i+1}^{n+1}}{\Delta x^2}.$$

Collecting the new temperatures on the left gives

$$\boxed{-\mathrm{Fo}T_{i-1}^{n+1} + (1 + 2\mathrm{Fo})T_i^{n+1} - \mathrm{Fo}T_{i+1}^{n+1} = T_i^n.} \tag{3}$$

The new neighbors are unknown too. Solve all these equations together as $A\mathbf{T}^{n+1} = \mathbf{b}$. For a five-node illustration, including the two fixed endpoints, let $d = 1 + 2\mathrm{Fo}$:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ -\mathrm{Fo} & d & -\mathrm{Fo} & 0 & 0 \\ 0 & -\mathrm{Fo} & d & -\mathrm{Fo} & 0 \\ 0 & 0 & -\mathrm{Fo} & d & -\mathrm{Fo} \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} T_0^{n+1} \\ T_1^{n+1} \\ T_2^{n+1} \\ T_3^{n+1} \\ T_4^{n+1} \end{bmatrix} = \begin{bmatrix} T_{\mathrm{left}} \\ T_1^n \\ T_2^n \\ T_3^n \\ T_{\mathrm{right}} \end{bmatrix}.$$

The matrix is tridiagonal: only its main diagonal and the two neighboring diagonals can contain nonzero entries. The example stores it as an ordinary NumPy array for readability.

```
A = np.zeros((number_of_nodes, number_of_nodes))
for i in range(1, number_of_nodes - 1):
    A[i, i - 1] = -Fo
    A[i, i] = 1.0 + 2.0 * Fo
    A[i, i + 1] = -Fo
A[0, 0] = 1.0
A[-1, -1] = 1.0


for n in range(number_of_steps):
    b = T_history[n, :].copy()
    b[0] = T_left
    b[-1] = T_right
    T_history[n + 1, :] = np.linalg.solve(A, b)
```

`A[i, j]` is the coefficient of unknown j in equation i. `copy()` makes an independent array so changing `b` does not overwrite the old temperature row. `np.linalg.solve(A, b)` returns the entire new row. There is no need to calculate a matrix inverse.

5. Compare methods and recognize fixed boundaries

Feature	Explicit FTCS	Implicit Backward Euler
Neighbors used	Old time n	New time $n+1$
Work each step	Direct temperature updates	Simultaneous linear solve
Stability for the original problem	Requires Fo ≤ 0.5	Stable for any positive Δt
Interior accuracy for smooth solutions	First order in time; second order in space	First order in time; second order in space

Unconditional stability does not mean unconditional accuracy. A large implicit time step can smooth away a rapid transient. Both methods should approach the same physical solution as space and time steps are reduced. The sudden initial change at the left end is not smooth, so early-time errors can be larger than later-time errors.

The printed time measures solver setup and time advancement, excluding imports, shared grid/history allocation, and plotting. The explicit code uses Python loops; the implicit code calls a dense NumPy solver at every step. These timings compare the supplied implementations. A specialized tridiagonal algorithm would change the implicit cost substantially.

Dirichlet means the temperature itself is prescribed

The original boundary equations are

$$T_0^n = T_{\text{left}}, \qquad T_N^n = T_{\text{right}}.$$

In both scripts, the initialization block is

```
T_history[0, :] = T_initial
T_history[:, 0] = T_left
T_history[:, -1] = T_right
```

The last two lines fill entire columns. Thus the endpoint values override the uniform initial temperature at time zero. Physically, the left surface is suddenly raised to 100 °C, while the interior starts at 20 °C.

The explicit loop leaves these columns alone. The implicit code also enforces them with identity matrix rows and the right-hand-side values `b[0]` and `b[-1]`. To change a fixed right boundary, remove *all* of its fixed-temperature enforcement, not just one line.

A safe editing workflow

Make a copy of the appropriate file and choose one right-boundary recipe on the following pages. Replace the identified statements instead of appending a second solver loop. Keep the grid, interior equations, timing statements, and plotting section. A line beginning with `##` is a comment; Python ignores it. Pasting an equation into a comment explains it but does not change the calculation.

6. Boundary physics: flux, insulation, and convection

Take x positive from left to right. At the right end, positive outward heat flux removes heat from the rod:

$$-k \left.\frac{\partial T}{\partial x}\right|_L = q_{\text{out}}.$$

A negative q_{out} adds heat there. Insulation is the special case $q_{\text{out}} = 0$. A specified flux determines a temperature *gradient*; it does not fix the surface temperature.

For convection at the right end,

$$q_{\text{out}} = h(T_N - T_\infty), \qquad \text{Bi} = \frac{h\Delta x}{k}.$$

Here $h \geq 0$ is the convection coefficient in $\text{W}/(\text{m}^2\,\text{K})$, and T_∞ is the surrounding fluid temperature. This Bi is a *mesh* Biot number based on Δx. A surface hotter than the fluid loses heat; a colder surface gains heat.

Where the factor of two comes from

Introduce a fictitious node $N+1$ just outside the right surface. A centered approximation to the flux gives

$$-k\frac{T_{N+1} - T_{N-1}}{2\Delta x} = q_{\text{out}} \quad \Longrightarrow \quad T_{N+1} = T_{N-1} - \frac{2\Delta x}{k} q_{\text{out}}.$$

Substituting this into the surface heat equation gives

$$\frac{dT_N}{dt} = \frac{2\alpha}{\Delta x^2}\left(T_{N-1} - T_N - \frac{\Delta x}{k} q_{\text{out}}\right). \tag{4}$$

The same expression follows from an energy balance on a half-width end cell. Per unit cross-sectional area, its heat capacity is $\rho c_p \Delta x/2$, and its net heat input is $k(T_{N-1} - T_N)/\Delta x - q_{\text{out}}$. The factor of two reflects that smaller end-cell volume. The ghost node is eliminated algebraically; no extra array element is required.

Two choices of time level

For explicit FTCS, evaluate the parentheses in Equation (4) at time n. For Backward Euler, evaluate them at $n+1$ and move unknown temperatures to the matrix side. Insulation therefore does *not* require overwriting the surface temperature with the adjacent node's value: the surface retains heat capacity and receives its own update.

7. Recipe A: specified right heat flux or insulation

Inputs for either method. Near the other physical inputs, add:

```
q_flux = 1000.0  # W/m^2, positive OUT of the right end
```

Use `q_flux = 0.0` for insulation. For the modified right surface, delete `T_history[:, -1] = T_right` in the initialization block. Keep `T_history[0, :] = T_initial` and `T_history[:, 0] = T_left`. The initial right temperature is now `T_initial`; its later values will be calculated.

Explicit file: replace its entire time loop with this block

From Equation (4),

$$T_N^{n+1} = T_N^n + 2\mathrm{Fo}\left(T_{N-1}^n - T_N^n - \frac{\Delta x}{k} q_{\mathrm{flux}}\right).$$

```
for n in range(number_of_steps):
    for i in range(1, number_of_nodes - 1):
        T_history[n + 1, i] = T_history[n, i] + Fo * (
            T_history[n, i - 1]
            - 2.0 * T_history[n, i]
            + T_history[n, i + 1]
        )
    # Inside the time loop, OUTSIDE the spatial loop:
    T_history[n + 1, -1] = T_history[n, -1] + 2.0 * Fo * (
        T_history[n, -2] - T_history[n, -1]
        - dx * q_flux / k
    )
```

Keep `solver_start` before this loop and `solver_seconds` after it. The original $\mathrm{Fo} \leq 0.5$ check remains suitable for this specified-flux update.

Implicit file: change the last matrix row and its right side

The new surface equation is

$$-2\mathrm{Fo}T_{N-1}^{n+1} + (1 + 2\mathrm{Fo})T_N^{n+1} = T_N^n - 2\mathrm{Fo}\frac{\Delta x}{k} q_{\mathrm{flux}}.$$

Replace `A[-1, -1] = 1.0` before the time loop with

```
A[-1, :] = 0.0  # Clear the entire old boundary row.
A[-1, -2] = -2.0 * Fo
A[-1, -1] = 1.0 + 2.0 * Fo
```

Inside the time loop, replace `b[-1] = T_right` with the following line, indented four spaces like `b[0] = T_left`:

```
    b[-1] = T_history[n, -1] - 2.0 * Fo * dx * q_flux / k
```

Keep the interior matrix rows and the left boundary row as written. Inside the time loop, also keep `b = T_history[n, :].copy()` and the call to `np.linalg.solve(A,b)`.

8. Recipe B: convection at the right surface

Start from a copy with the original fixed boundaries, not a mixture of Recipes A and B. Add these inputs near the top:

```
h = 100.0          # W/(m^2 K), convection coefficient
T_infinity = 20.0  # deg C, surrounding fluid temperature
```

After `dx` and `Fo` have been calculated, add

```
if h < 0:
    raise ValueError("h must be nonnegative.")
Bi = h * dx / k
```

Delete `T_history[:, -1] = T_right` from initialization, leaving the right surface initially at `T_initial`.

Explicit update and its extra stability check

Substitute $q_{\text{out}}^n = h(T_N^n - T_\infty)$ into the flux update:

$$T_N^{n+1} = T_N^n + 2\text{Fo}\left[T_{N-1}^n - (1+\text{Bi})T_N^n + \text{Bi}T_\infty\right].$$

Use the full explicit loop from Recipe A, but replace its last surface assignment with this block (including the shown indentation):

```
    T_history[n + 1, -1] = T_history[n, -1] + 2.0 * Fo * (
        T_history[n, -2] - (1.0 + Bi) * T_history[n, -1]
        + Bi * T_infinity
    )
```

Do not also use `q_flux` at this boundary. The coefficient of the old surface temperature is $1 - 2\text{Fo}(1+\text{Bi})$. A sufficient stability restriction giving nonnegative update coefficients is

$$\text{Fo} \le \frac{1}{2(1+\text{Bi})}.$$

After defining `Bi`, add the following check before the time loop. The existing interior check can stay in place.

```
Fo_limit = 1.0 / (2.0 * (1.0 + Bi))
if Fo > Fo_limit:
    raise ValueError("Reduce dt for the convection boundary.")
```

For the example $h = 100$ and the current fine grid, $\text{Bi} \approx 0.001111$ and the limit is approximately 0.499445, above the chosen Fo.

Implicit matrix row and right-hand side

Using $q_{\text{out}}^{n+1} = h(T_N^{n+1} - T_\infty)$ gives

$$-2\text{Fo}T_{N-1}^{n+1} + [1 + 2\text{Fo}(1+\text{Bi})]T_N^{n+1} = T_N^n + 2\text{FoBi}T_\infty.$$

Replace the original `A[-1, -1] = 1.0` with

```
A[-1, :] = 0.0
A[-1, -2] = -2.0 * Fo
A[-1, -1] = 1.0 + 2.0 * Fo * (1.0 + Bi)
```

Replace `b[-1] = T_right` inside the time loop with

```
    b[-1] = T_history[n, -1] + 2.0 * Fo * Bi * T_infinity
```

For constant h, the matrix is still built once. Backward Euler does not need the explicit convection time-step restriction for this linear problem.

9. What changes at the left end?

The outward direction is $-x$ at the left end. Therefore, with outward heat flux positive,

$$q_{\text{out,left}} = +k \left.\frac{\partial T}{\partial x}\right|_0, \qquad +k \left.\frac{\partial T}{\partial x}\right|_0 = h_{\text{left}}(T_0 - T_{\infty,\text{left}}).$$

Using this outward convention, the right-surface formulas apply at the left with the surface index N replaced by 0 and its neighbor $N-1$ replaced by 1. Positive outward flux always removes heat.

For example, to replace a fixed *left* end with specified outward flux, define `q_left` near the inputs, delete `T_history[:, 0] = T_left`, and initialize the left surface with `T_initial`. In the explicit time loop, outside its inner loop, use

```
    T_history[n + 1, 0] = T_history[n, 0] + 2.0 * Fo * (
        T_history[n, 1] - T_history[n, 0] - dx * q_left / k
    )
```

In the implicit code, replace `A[0, 0] = 1.0` with

```
A[0, :] = 0.0
A[0, 0] = 1.0 + 2.0 * Fo
A[0, 1] = -2.0 * Fo
```

and replace `b[0] = T_left` with

```
    b[0] = T_history[n, 0] - 2.0 * Fo * dx * q_left / k
```

For left convection, define `Bi_left = h_left * dx / k`. The explicit parentheses become $T_1^n - (1 + \text{Bi}_{\text{left}})T_0^n + \text{Bi}_{\text{left}}T_{\infty,\text{left}}$. The implicit main-diagonal coefficient becomes $1 + 2\text{Fo}(1 + \text{Bi}_{\text{left}})$ and $b_0 = T_0^n + 2\text{FoBi}_{\text{left}}T_{\infty,\text{left}}$. If both ends convect, apply the explicit restriction for both ends, using the smaller allowable Fourier number.

Do not silently change flux conventions. If a problem instead defines left flux by $-k\,\partial T/\partial x = q_x$, positive q_x points into the rod. In that case $q_{\text{out,left}} = -q_x$, so the left flux contribution has a plus sign. State the convention before editing.

10. Keep the contour colors consistent with the new problem

After changing a boundary, `T_right` or `T_left` may no longer control that surface. Flux or a different ambient temperature may also produce temperatures outside the original 20–100 °C range. In the plotting section, replace the two original min/max lines by

```
T_min = float(np.min(T_history))
T_max = float(np.max(T_history))
```

Keep the uniform-temperature fallback, `levels`, `contourf`, and PNG saving code. This covers all computed values. For side-by-side method comparisons, choose identical color limits covering both results. The plot's vertical axis is *time*, not a second spatial direction.

11. Checks that tell you whether an edit makes sense

Check against a steady solution

At steady state, the temperature is a straight line because $d^2T/dx^2 = 0$. With left temperature fixed, the following targets are useful:

$$\begin{aligned}
\text{Fixed right temperature:} \quad & T(x) = T_{\text{left}} + (T_{\text{right}} - T_{\text{left}})\frac{x}{L}, \\
\text{Specified right outward flux:} \quad & T(x) = T_{\text{left}} - \frac{q_{\text{flux}}}{k}x, \\
\text{Right convection:} \quad & q_{\text{steady}} = \frac{T_{\text{left}} - T_\infty}{L/k + 1/h}, \qquad h > 0, \\
& T(x) = T_{\text{left}} - \frac{q_{\text{steady}}}{k}x.
\end{aligned}$$

For an insulated right end, the eventual temperature is uniformly T_{left}. For the example flux 1000 W/m^2, the steady right temperature is approximately 97.78 °C. For $h = 100$ W/(m^2 K) and $T_\infty = 20$ °C, it is approximately 85.45 °C. These are *long-time* targets, not temperatures guaranteed at 300 s. Increase `total_time` and check whether later profiles stop changing.

Simple experiments

1. Set both fixed temperatures and the initial temperature equal. Both methods should retain a uniform temperature at every time.
2. Set the right flux to zero with the left end held hot. The right temperature should evolve; it should not stay fixed at its old value.
3. Set $h = 0$ in the convection recipe. Its equations should become exactly the insulated-boundary equations.
4. Use a positive right outward flux. Reversing its sign should reverse the direction of that boundary's imposed heat transfer.
5. Reduce Δx and Δt consistently in both methods. Compare temperatures at the same physical position and time; their difference should decrease as the solution is resolved.

Common beginner errors and their fixes

The new boundary remains fixed. Remove its whole-column assignment. In the implicit file, also replace its matrix row and its `b` entry.

IndentationError. Use spaces consistently. A boundary update belongs inside the time loop, at the same indentation as `for i`, not inside the spatial loop. An implicit `b` change belongs before the solve.

NameError. Define new inputs before using them, and define `Bi` after `dx`. Python runs the file from top to bottom.

Temperatures oscillate or explode. Recheck the explicit time-step restriction, including convection, and make sure all right-side values use the old time row.

The image has blank temperature bands. Use the updated plotting limits from the preceding section so the levels cover the computed range.

Save the file after each edit. Read the last line of a Python error message first. It often identifies the problem and points to the relevant line.